Zero Copy Rx with io_uring

David Wei, Pavel Begunkov

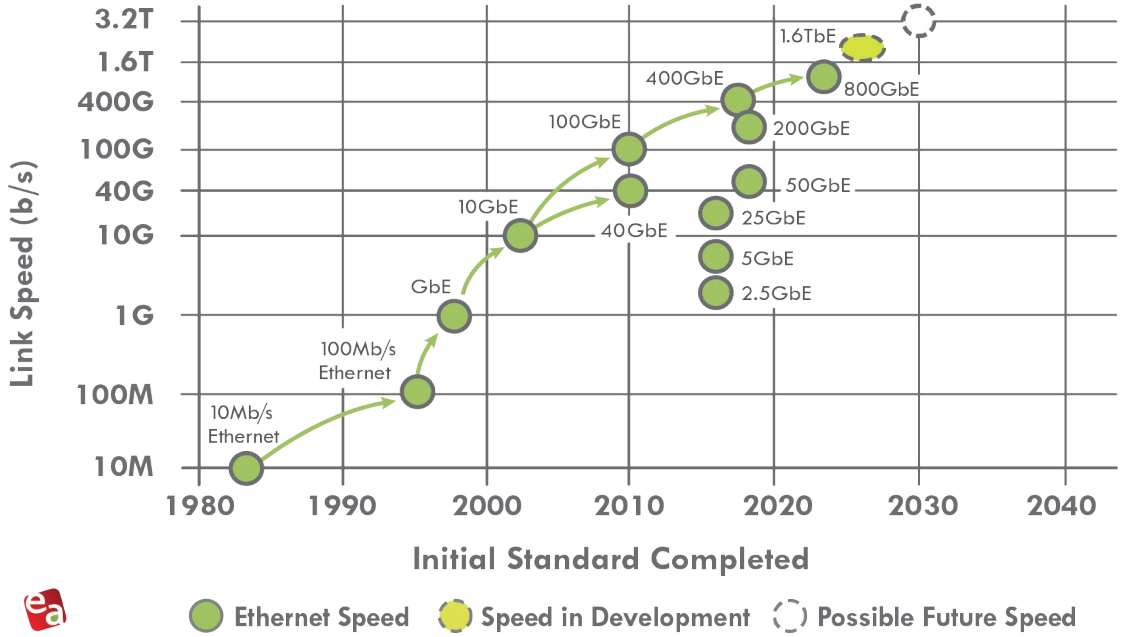
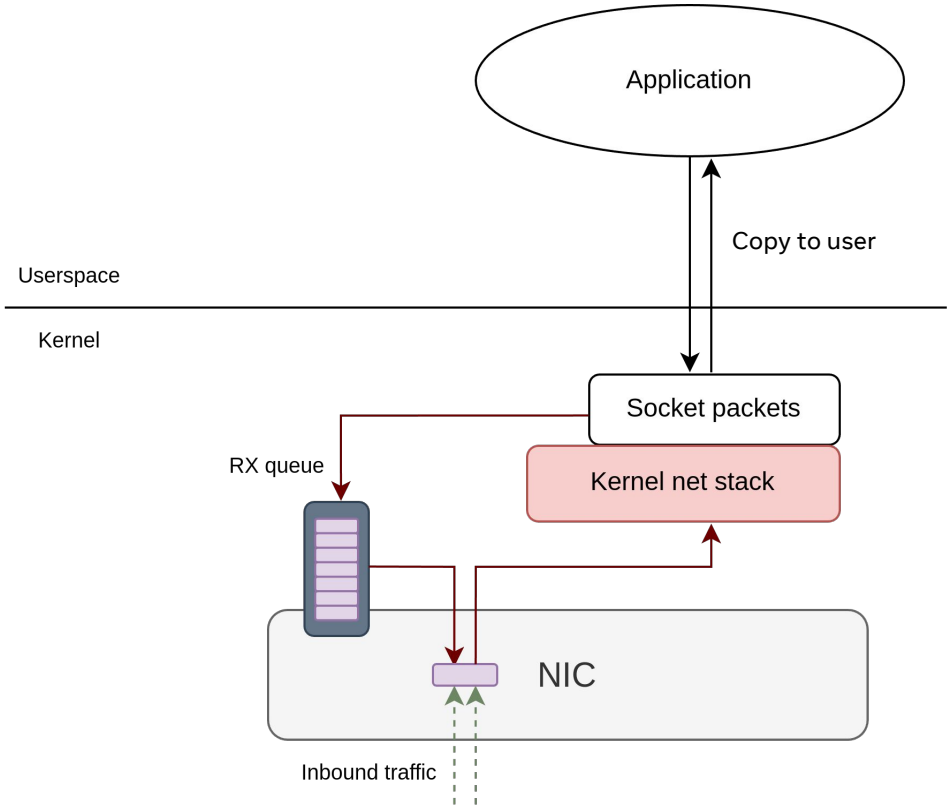
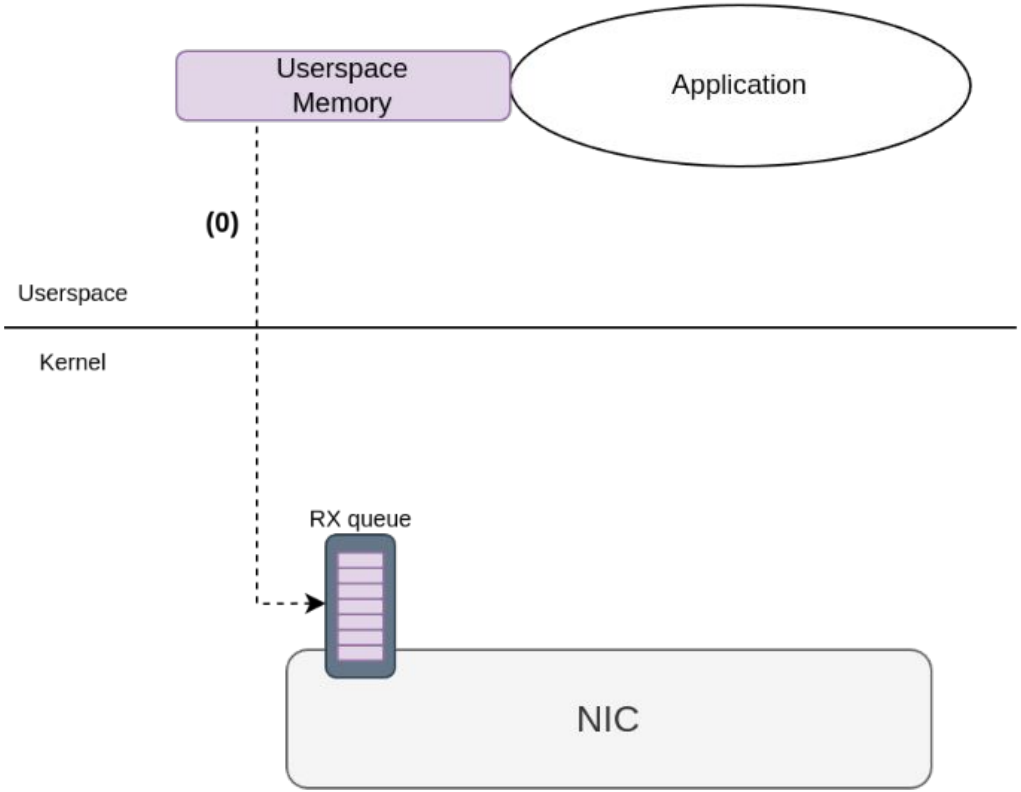


image credit: Ethernet Alliance
https://ethernetalliance.org/technology/ethernet-roadmap/

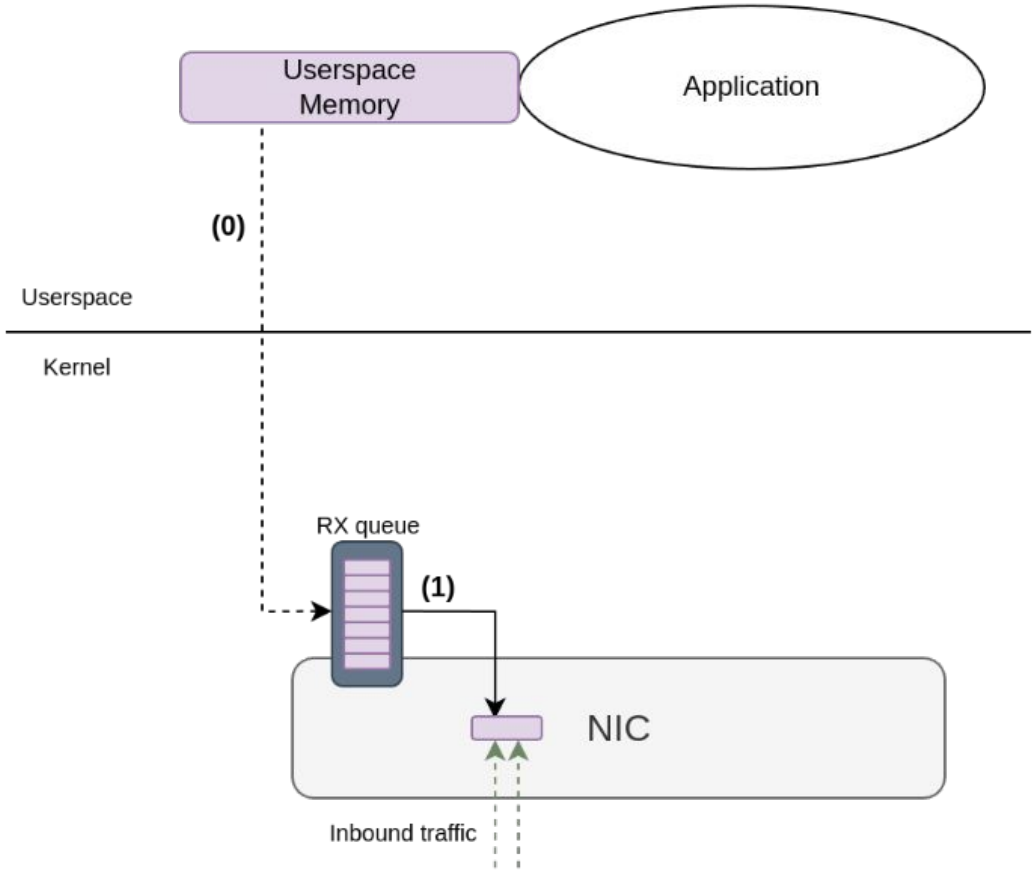
Zerocopy net is not new to Linux

- TCP_ZEROCOPY_RECEIVE
- DPDK
- AF_XDP
- InfiniBand

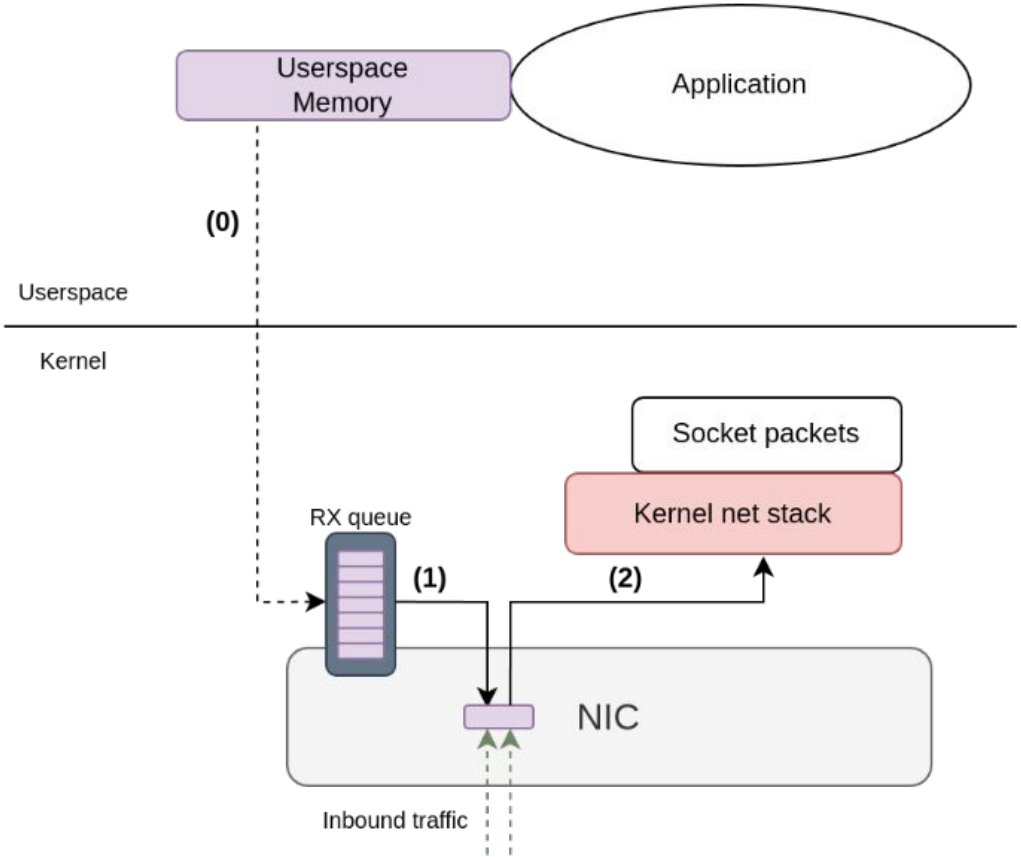

Application
Copy to user
Userspace
Kernel
Socket packets
RX queue
Kernel net stack
NIC
Inbound traffic



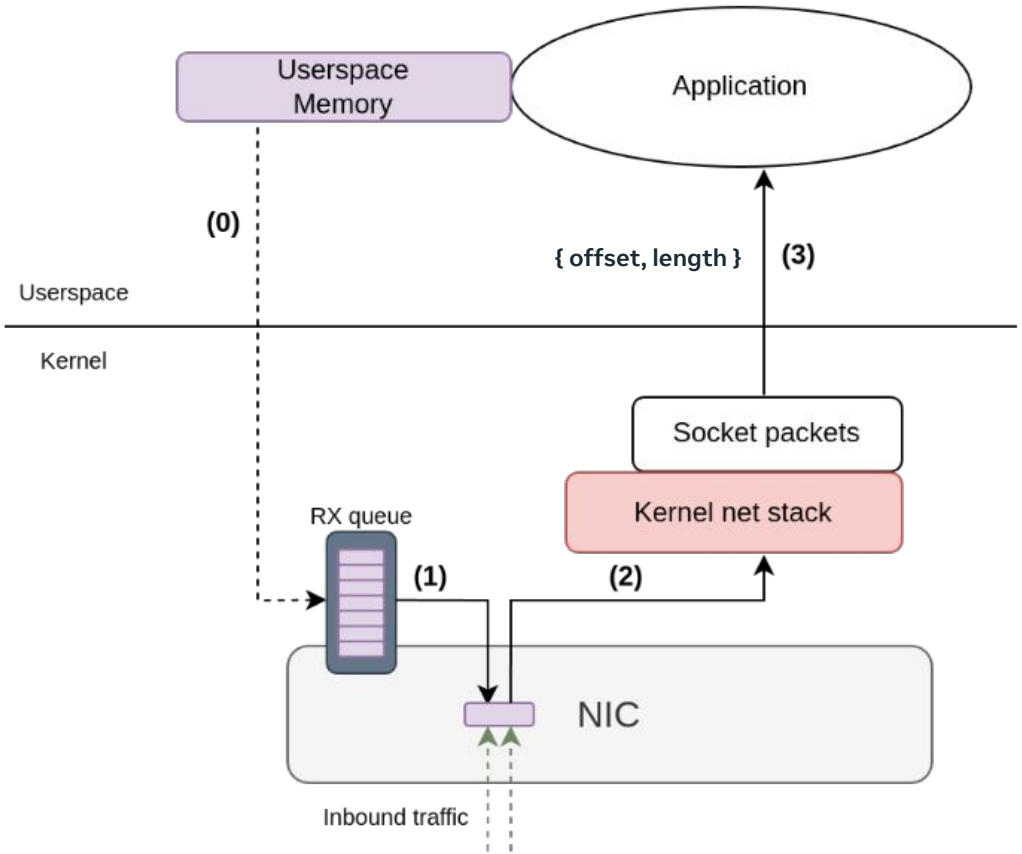
(0) populate rx ring with user memory



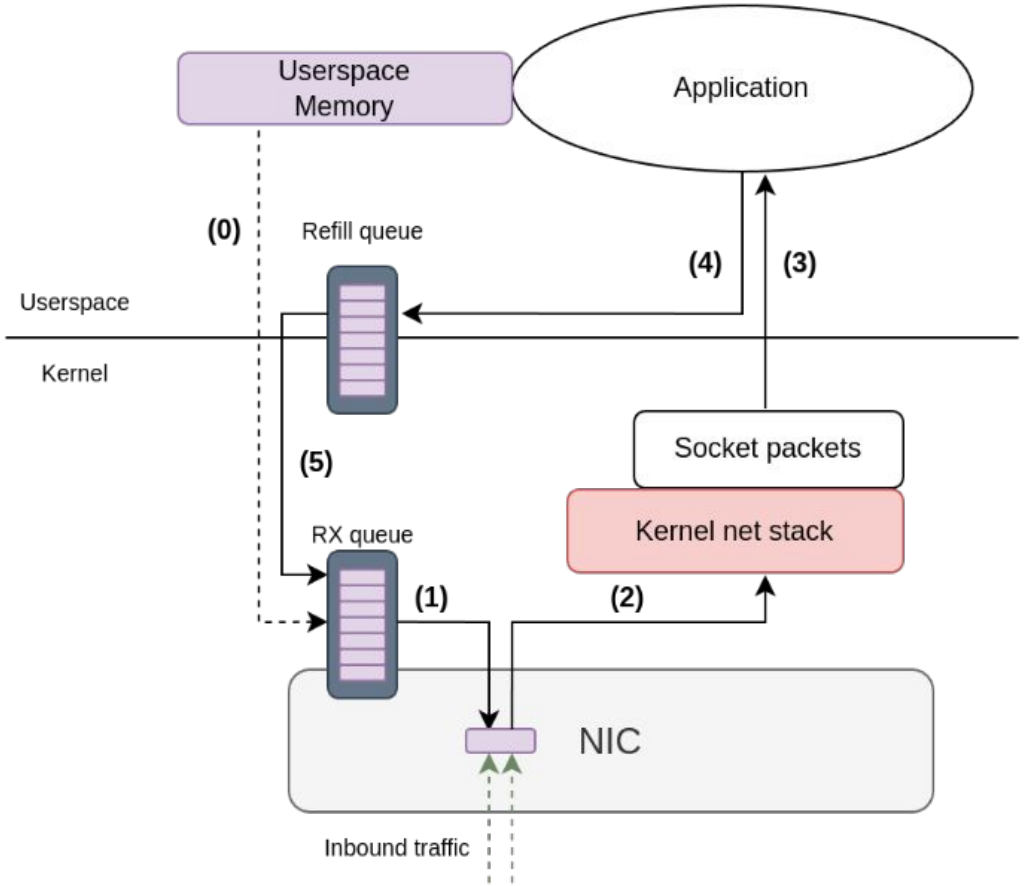
(0) populate rx ring with user memory
(1) pick a buffer and receive data



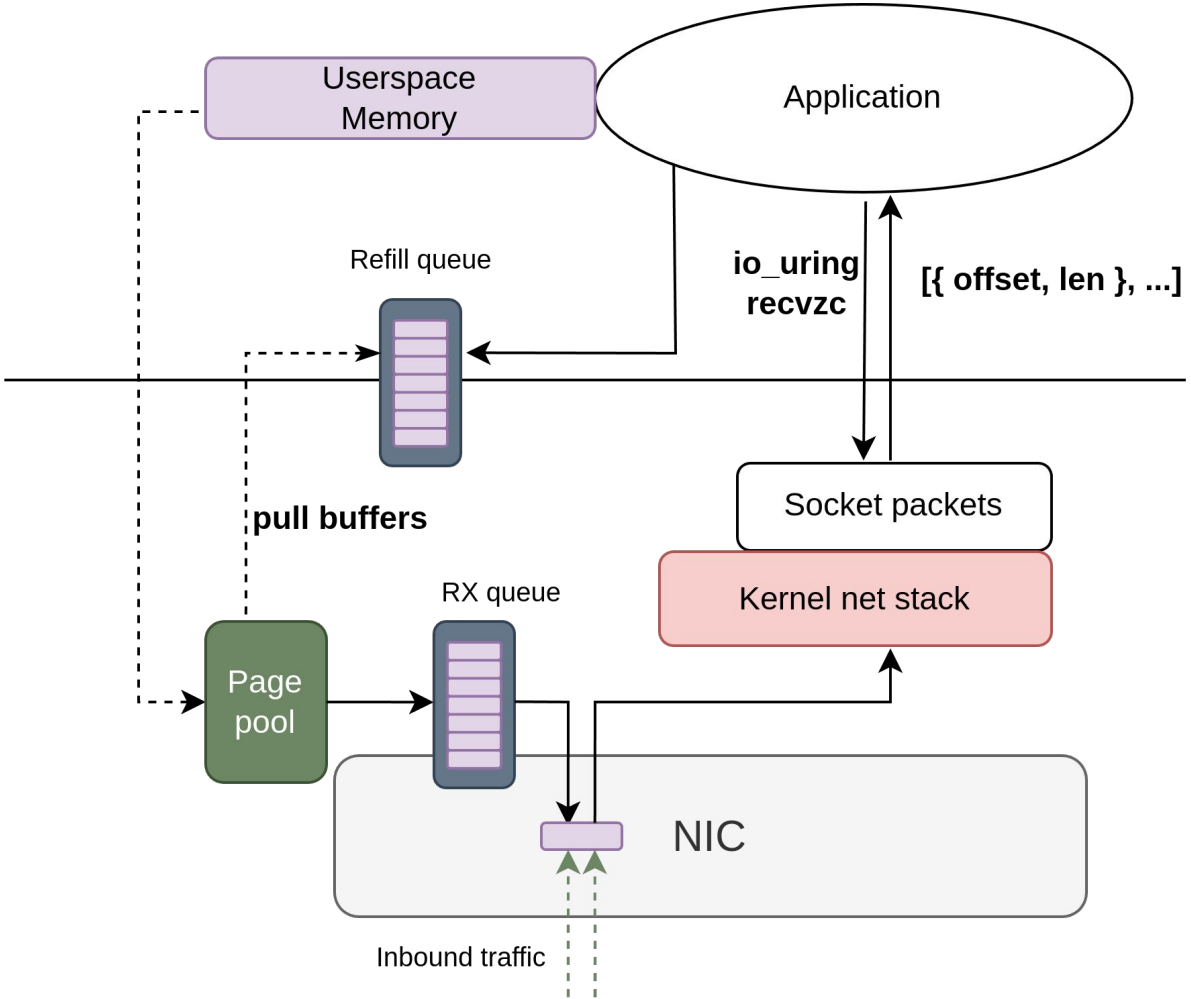
(0) populate rx ring with user memory
(1) pick a buffer and receive data
(2) pass the buffer to the net stack



(0) populate rx ring with user memory
(1) pick a buffer and receive data
(2) pass the buffer to the net stack
(3) give the buffer to the user



(0) populate rx ring with user memory
(1) pick a buffer and receive data
(2) pass the buffer to the net stack
(3) give the buffer to the user
(4) when user is done with the buffer, pass it back via the refill queue
(5) return the buffer to the NIC


Userspace Memory
Application
Refill queue
io_uring recvzc
[{ offset, len }, ...]
pull buffers
Socket packets
Kernel net stack
RX queue
Page pool
NIC
Inbound traffic

Performance

Benchmark setup

- AMD EPYC 9454
  - No DDIO
- Broadcom BCM957508 200G
  - ✅ HW GRO
  - ✅ IOMMU
- Kernel v6.11 base

kperf

- More sophisticated iperf
- memcmp payload pattern check
- Single TCP connection
- Single sender + receiver worker thread
- Net softirq for the given connection pinned to separate CPU core

Results: 1500 MTU

MTU	memcmp	Engine	BW	Gain	Net CPU busy%	Net CPU softirq%
1500	✅	epoll	68.8 Gbps		28.4%	27.9%
1500	✅	io_uring ZC	90.4 Gbps	+31.4%	37.2%	35.7%
1500	❌	epoll	74.4 Gbps		35.7%	34.7%
1500	❌	io_uring ZC	106.7 Gbps	+43.4%	47.9%	46.4%

Results: 4096 MTU

MTU	memcmp	Engine	BW	Gain	Net CPU busy%	Net CPU softirq%
4096	✅	epoll	66.9 Gbps		24.8%	23.7%
4096	✅	io_uring ZC	92.2 Gbps	+37.8%	36.6%	35.0%
4096	❌	epoll	82.2 Gbps		33.2%	32.6%
4096	❌	io_uring ZC	116.2 Gbps	+41.4%	48.2%	46.6%

Results: worker/softirq same CPU

MTU	memcmp	Engine	BW	Gain	CPU busy%	CPU softirq%
1500	❌	epoll	62.6 Gbps		100%	17.8%
1500	❌	io_uring ZC	80.9 Gbps	+29.2%	100%	45.6%

Networking

netmem_ref

- Three memory types in the networking stack:
  - struct page: Host kernel memory
  - struct net_iov:
    - Host userspace memory (io_uring ZC Rx)
    - Device memory (TCP devmem)
- Introduce new abstraction netmem_ref

```
/**
 * typedef netmem_ref - a nonexistent type marking a reference to generic
 * network memory.
 */
typedef unsigned long __bitwise netmem_ref;
```

net_iov

- The lifetime of pages must be managed by us and not the networking stack
- TCP devmem is not even passing pages
- Some abstraction is required → struct net_iov

```c
/*  We overload the LSB of the struct page pointer to indicate whether it's
 *  a page or net_iov.
 */
#define NET_IOV 0x01UL

struct net_iov {
    unsigned long __unused_padding;
    unsigned long pp_magic;
    struct page_pool *pp;
    struct net_iov_area *owner;
    unsigned long dma_addr;
    atomic_long_t pp_ref_count;
};
```

Netdev Queue API Context

- All-or-nothing: queues are configured in the same way at netdev bringup
- Most configuration changes requires full device reset
- Memory allocations *after* netdev is brought down

Netdev Queue API Problems for ZC Rx

- Specific queues are configured for ZC Rx via page pool
  - Each time this happens → full netdev reset 😕
- The lifetime of flow steering rules for ZC Rx are not tied to the queue
  - Must be managed separately
- Want to have a *queue API* where queues are first-class kernel objects
  - Can be created/destroyed dynamically
  - Individual configuration e.g. page pool, descriptor size
  - Group lifetimes together e.g. flow steering and RSS contexts
  - Queue-by-queue reset model

netdev_queue_mgmt_ops

```
struct netdev_queue_mgmt_ops {
    size_t      ndo_queue_mem_size;
    int         (*ndo_queue_mem_alloc)(struct net_device *dev,
                                       void *per_queue_mem,
                                       int idx);
    void        (*ndo_queue_mem_free)(struct net_device *dev,
                                      void *per_queue_mem);
    int         (*ndo_queue_start)(struct net_device *dev,
                                   void *per_queue_mem,
                                   int idx);
    int         (*ndo_queue_stop)(struct net_device *dev,
                                  void *per_queue_mem,
                                  int idx);
};
```

netdev_rx_queue_restart()

```c
int netdev_rx_queue_restart(struct net_device *dev, unsigned int rxq_idx)
{
    struct netdev_rx_queue *rxq = __netif_get_rx_queue(dev, rxq_idx);

    new_mem = kvzalloc(dev->queue_mgmt_ops->ndo_queue_mem_size, GFP_KERNEL);
    old_mem = kvzalloc(dev->queue_mgmt_ops->ndo_queue_mem_size, GFP_KERNEL);

    dev->queue_mgmt_ops->ndo_queue_mem_alloc(dev, new_mem, rxq_idx);
    dev->queue_mgmt_ops->ndo_queue_stop(dev, old_mem, rxq_idx);
    dev->queue_mgmt_ops->ndo_queue_start(dev, new_mem, rxq_idx);
    dev->queue_mgmt_ops->ndo_queue_mem_free(dev, old_mem);
}
```

Vendor support

	netmem	Queue API	ZC Rx
Broadcom bnxt	✅(*)	✅	✅(*)
Google gve	✅	✅	✅
Mellanox mlx5	🚧(**)	🚧(**)	🚧

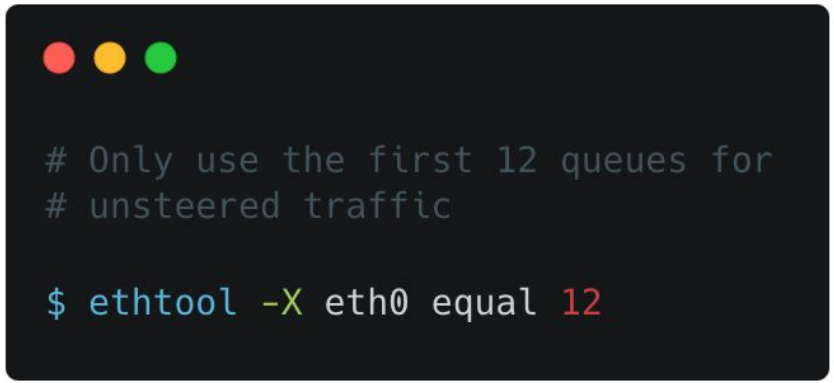
- (*) Support in our first patchset
- (**) Mellanox are working on these

User API

Setup: NIC


Queue management
Set eth0 to 24 combined Rx + Tx queues
$ ethtool -L eth0 combined 24

Setup: NIC


Only use the first 12 queues for
unsteered traffic
$ ethtool -X eth0 equal 12

Setup: NIC


Steer TCP flows with dst port == 9999 to queue 13
$ ethtool -N eth0 flow-type tcp6 \
dst-port 9999 \
action 12

Setup: io_uring

```
size_t area_size = PAGE_SIZE * entries;
void *area = aligned_alloc(PAGE_SIZE, area_size);
```

Setup: io_uring

```
size_t area_size = PAGE_SIZE * entries;
void *area = aligned_alloc(PAGE_SIZE, area_size);

struct io_uring_zcrx_area_reg area_reg = {
    .addr = (__u64)(unsigned long)area,
    .len = area_size,
    .flags = 0,
    .area_id = 0,
};
```

Setup: io_uring

```
size_t area_size = PAGE_SIZE * entries;
void *area = aligned_alloc(PAGE_SIZE, area_size);

struct io_uring_zcrx_area_reg area_reg = {
    .addr = (__u64)(unsigned long)area,
    .len = area_size,
    .flags = 0,
    .area_id = 0,
};

struct io_uring_zcrx_ifq_reg reg = {
    .if_idx = ifindex,
    .if_rxq = 12,
    .rq_entries = 4096,
    .area_ptr = (__u64)(unsigned long)&area_reg,
};
```

Setup: io_uring

```
size_t area_size = PAGE_SIZE * entries;
void *area = aligned_alloc(PAGE_SIZE, area_size);

struct io_uring_zcrx_area_reg area_reg = {
    .addr = (__u64)(unsigned long)area,
    .len = area_size,
    .flags = 0,
    .area_id = 0,
};

struct io_uring_zcrx_ifq_reg reg = {
    .if_idx = ifindex,
    .if_rxq = 12,
    .rq_entries = 4096,
    .area_ptr = (__u64)(unsigned long)&area_reg,
};

io_uring_register_ifq(ring, &reg);
```

Setup: refill ring

```
struct io_uring_zcrx_rq {
    unsigned *khead;
    unsigned *ktail;
    struct io_uring_zcrx_rqe *rqes;
    unsigned rq_tail;
    void *ring_ptr;
    unsigned ring_entries;
};
```

Setup: refill ring

```
struct io_uring_zcrx_rq {
    unsigned *khead;
    unsigned *ktail;
    struct io_uring_zcrx_rqe *rqes;
    unsigned rq_tail;
    void *ring_ptr;
    unsigned ring_entries;
};

io_uring_zcrx_mmap(ring, &reg, &rq);
```

Prepare request

```
struct io_uring_sqe *sqe;

sqe = io_uring_get_sqe(ring);
io_uring_prep_rw(IORING_OP_RECV_ZC, sqe, fd, NULL, 0, 0);
sqe->ioprio |= IORING_RECV_MULTISHOT;
```

Submit and wait

```
struct io_uring_sqe *sqe;

sqe = io_uring_get_sqe(ring);
io_uring_prep_rw(IORING_OP_RECV_ZC, sqe, fd, NULL, 0, 0);
sqe->ioprio |= IORING_RECV_MULTISHOT;

io_uring_submit_and_wait(ring, 1);
```

Process completions

```
struct io_uring_zcrx_cqe {
    __u64   off; // upper 16 bits is area_id
    __u64   __pad;
};

struct io_uring_cqe *cqe = get_cqe();
struct io_uring_zcrx_cqe *rcqe = (struct io_uring_zcrx_cqe *)(cqe + 1);

unsigned char *data = area_ptr + (rcqe->off & IORING_ZCRX_AREA_MASK);
```

Refill

```
struct io_uring_zcrx_rqe {
    __u64   off; // upper 16 bits is area_id
    __u32   len;
    __u32   __pad;
};

struct io_uring_zcrx_rqe *rqe;
rqe = &rq_ring.rqes[rq_ring.rq_tail & (rq_ring.ring_entries - 1)];

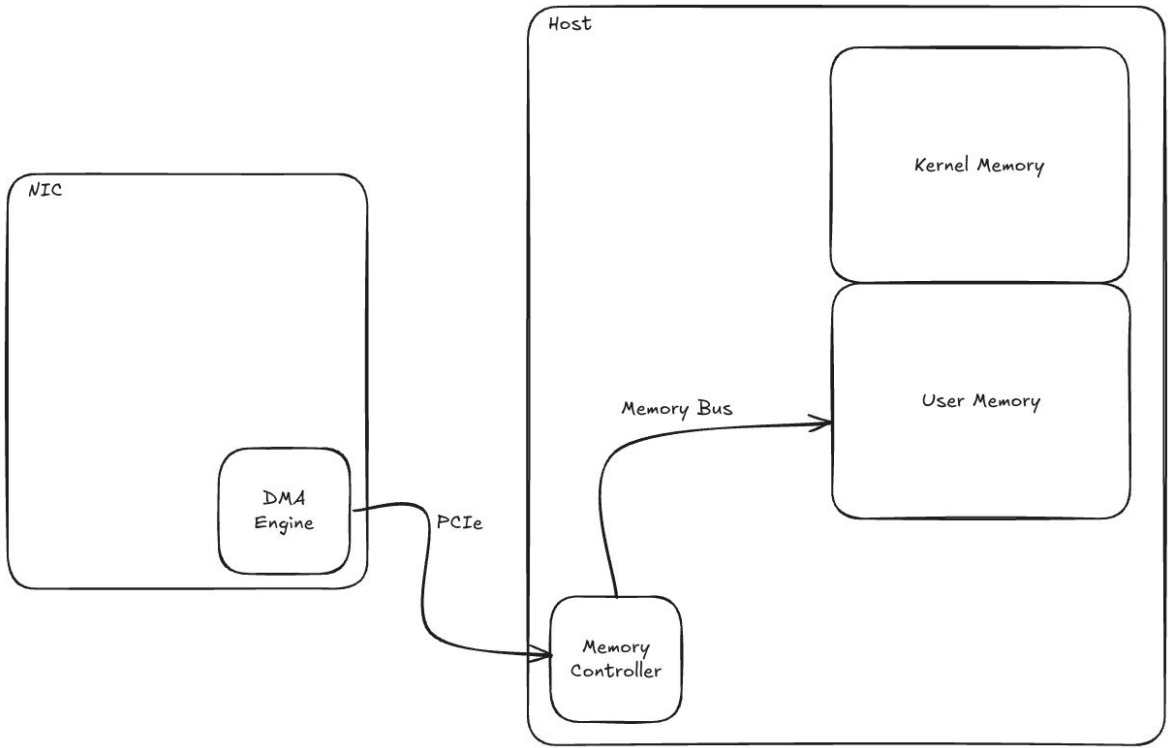
area_offset = rcqe->off & IORING_ZCRX_AREA_MASK;
rqe->off = area_offset | reg_area.rq_area_token;
rqe->len = cqe->res;
IO_URING_WRITE_ONCE(*rq_ring.ktail, ++rq_ring.rq_tail);
```

Userspace Challenges

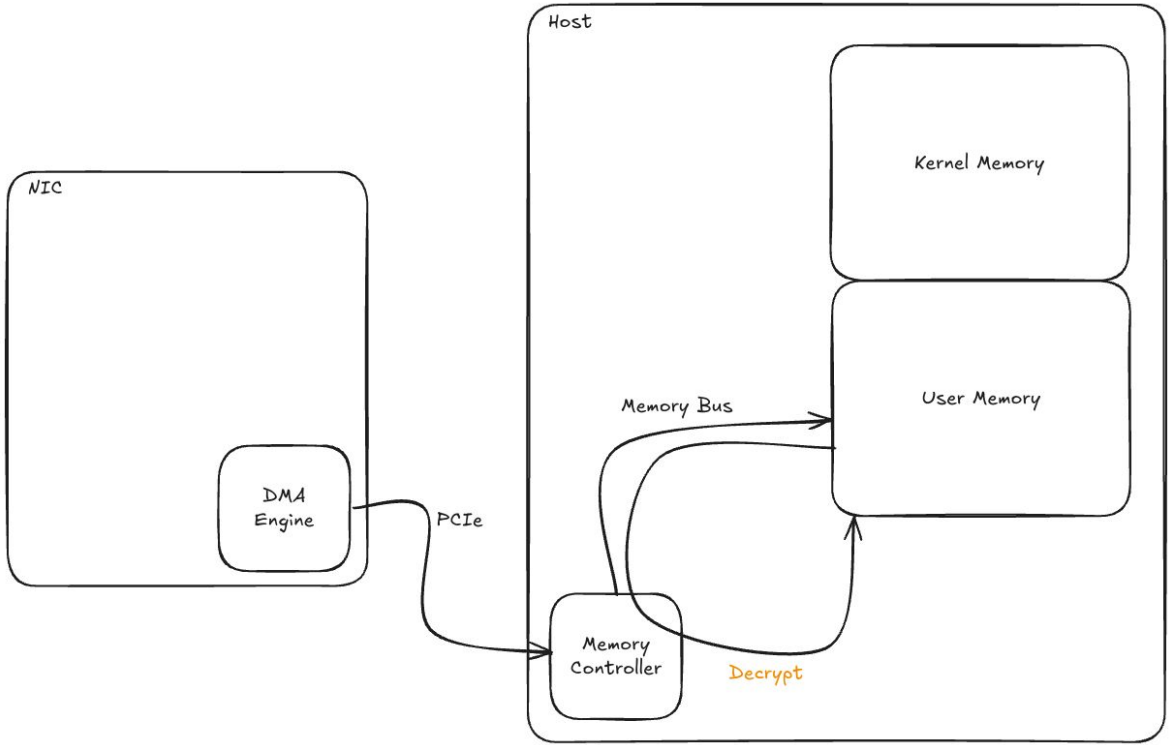
Avoiding user copies

- Removing a kernel → user copy is good
- But not if user has to add a copy back for some other reasons:
  - Decryption
  - Alignment
- …that could be combined with a copy

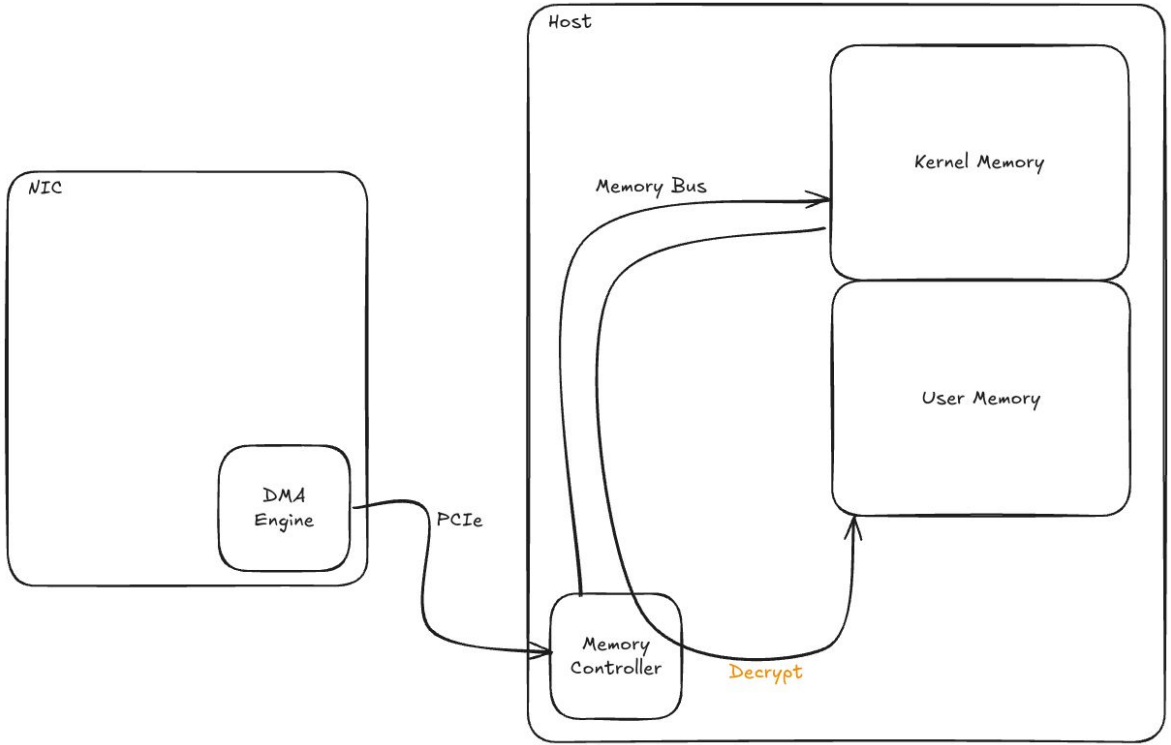
ZC + Plaintext



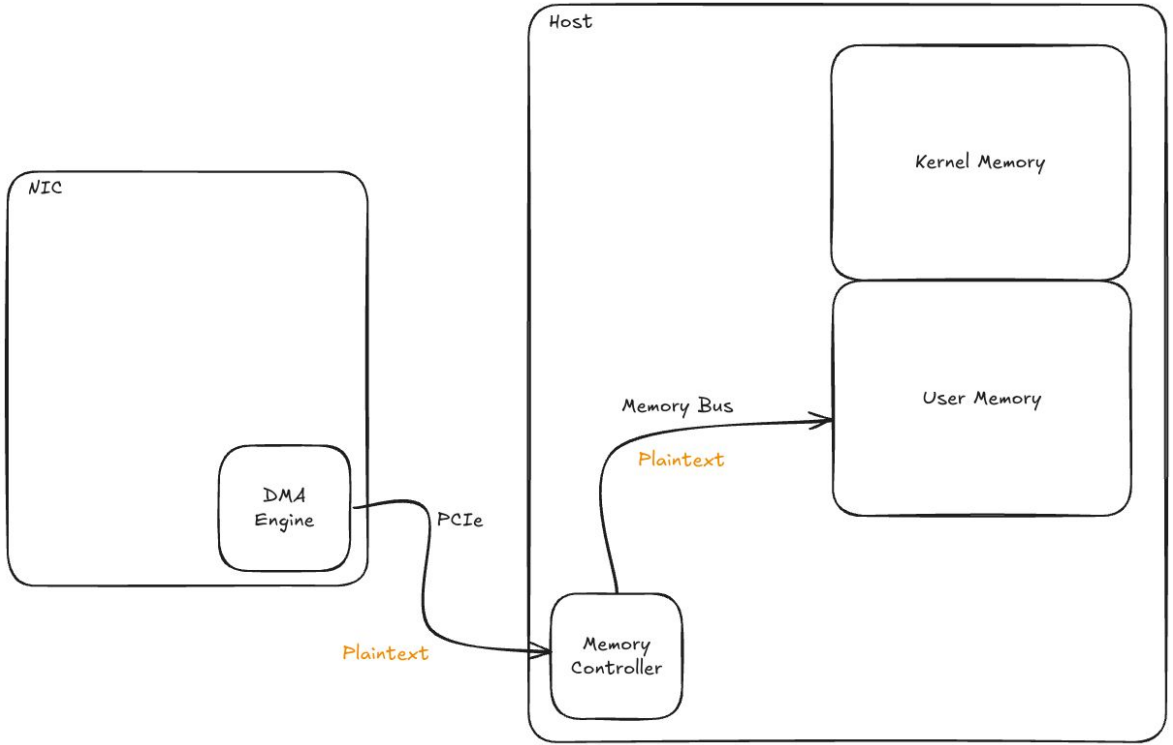
ZC + TLS


Host
Kernel Memory
NIC
User Memory
Memory Bus
DMA
Engine
PCIe
Memory
Controller
Decrypt

kTLS

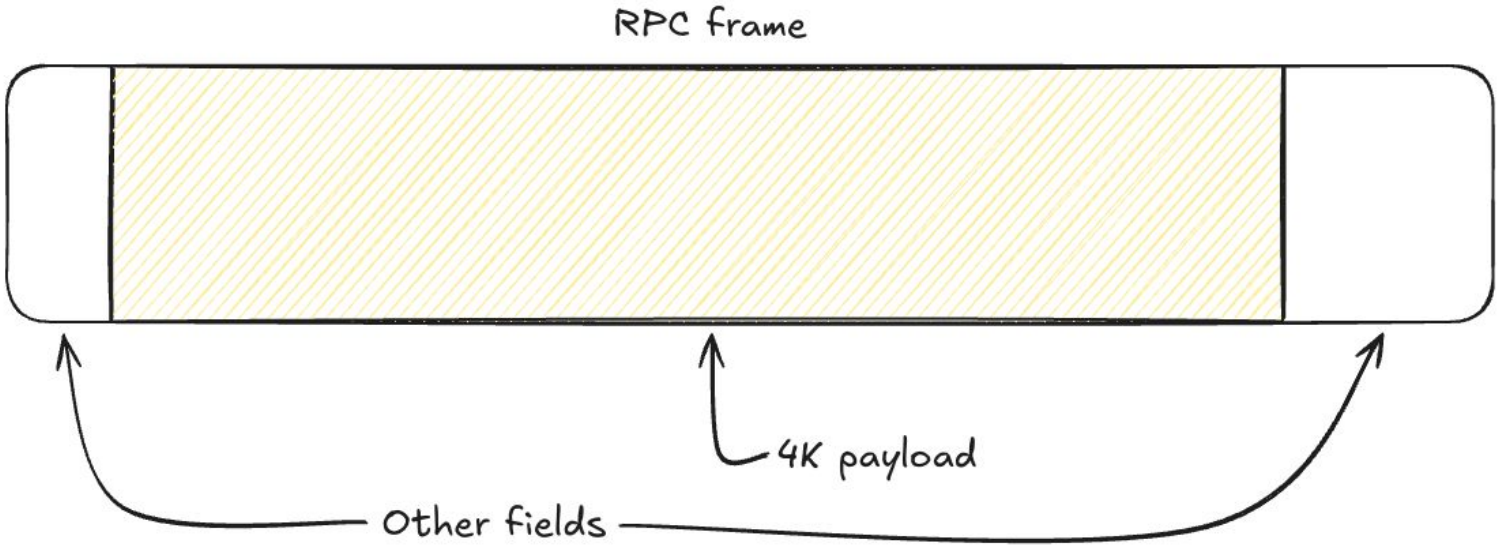

Host
NIC
Kernel Memory
Memory Bus
User Memory
DMA
Engine
PCIe
Memory
Controller
Decrypt

PSP

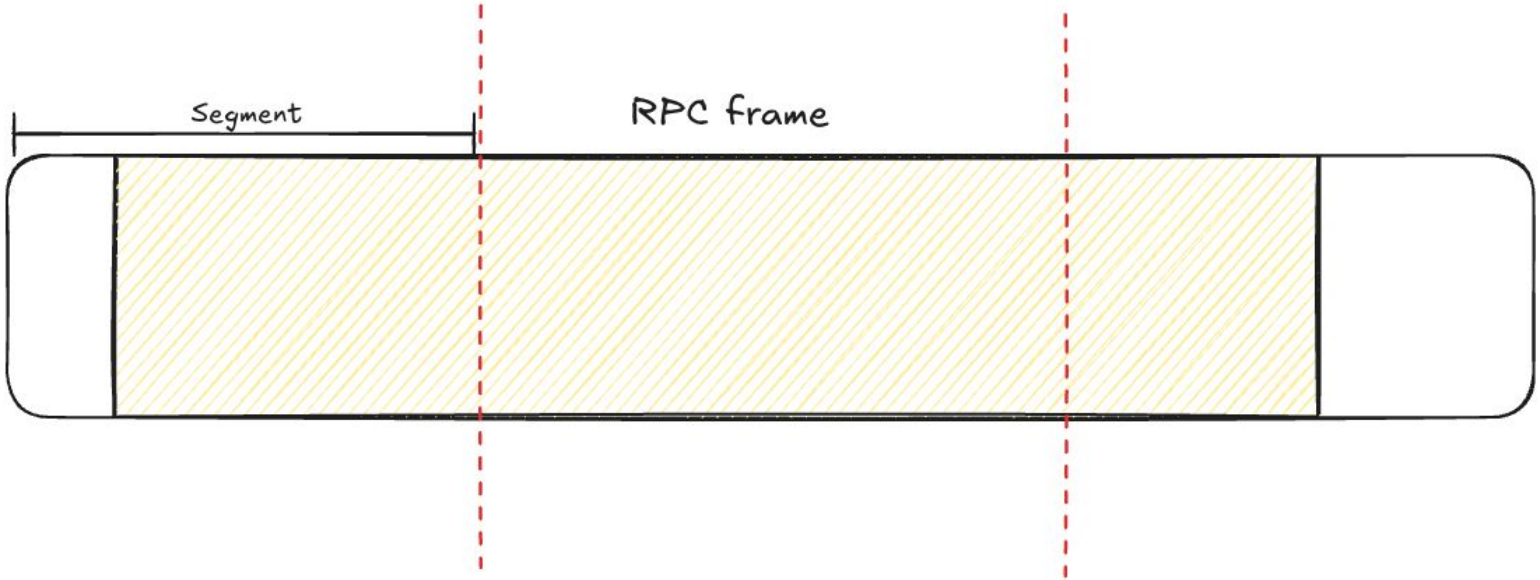

Host
Kernel Memory
NIC
User Memory
Memory Bus
Plaintext
DMA
Engine
PCIe
Plaintext
Memory
Controller

Alignment

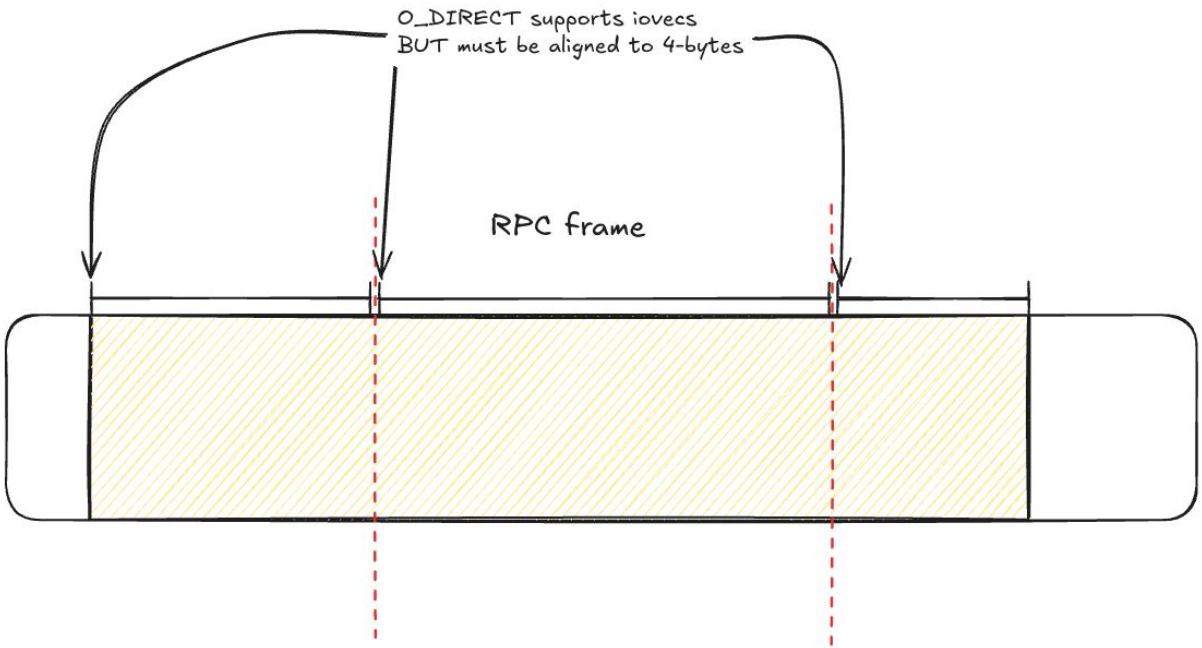
- Block storage service, 4 KB page writes, over RPC w/ framing



Alignment



Alignment


O_DIRECT supports iovecs
BUT must be aligned to 4-bytes
RPC frame

HW queue sharing

- Finite number of HW queues
- 1:1:1 HW queue:io_uring:thread association
- So CPUs with > 128 cores = 😢
- Something has to be shared

Future Work

Optimisations:

- Improving refcounting
- Support for huge pages and larger chunks

More features:

- Multiple areas
- dma-buf for p2p
- Some area sharing features

https://github.com/isilence/linux.git zcrx/v5-conf

Quite outdated RFC:

https://lore.kernel.org/io-uring/20240312214430.2923019-1-dw@davidwei.uk/

Benchmarking:

https://github.com/spikeh/netbench/tree/zcrx/next

https://github.com/spikeh/kperf/tree/zcrx/next

Contact us:

io-uring at vger.kernel.org
Pavel Begunkov <asml.silence@gmail.com>
David Wei <dw@davidwei.uk>